

Suivi de projet : TETRISTE

Préambule

Dans ce document, nous présenterons les différentes phases de ce projet, pour en analyser le fonctionnement et comprendre quelles ont été les défis dans l'implémentation. Nous tenterons d'expliciter également les moyens mis en place pour y faire face.

Bonne lecture !

Par Matéo DESCHILDRE et Damien ROY

I. Difficultés majeures rencontrées

Vous trouverez dans le dossier racine une présentation PowerPoint pour mieux pouvoir vous représentez les différentes fonctions du jeu ainsi que des explications complémentaires.

A partir du cahier des charges du projet, la première question que nous nous sommes posés concernait l'implémentation des règles du jeu. De nombreuses questions ont émergé après plusieurs lectures : quelles sont les différentes structures que nous devons utiliser, quelles seraient les fonctions de bases du projet, quelles sont les bibliothèques nécessaires...

Dans un premier temps, nous avons travaillé sur la définition des structures principales : « Piece » et « Plateau ». Dans la structure « Piece », nous trouvons : *forme*, *couleur*, *suivForme*, *precForme*, *suivCouleur*, *precCouleur*, *suiv*. Dans la structure « Plateau », nous trouvons : *premier*, *card*, et *tailleMax*. Nous n'avons pas souhaité créer plusieurs sous listes de formes et de couleurs, mais simplement l'introduire directement dans la structure de la pièce, ce qui facilite la compréhension et nous évite de dupliquer huit fois une même liste chaînée.

Après avoir implémenté cette première partie, nous avons débuté le développement du jeu, en commençant par remplir les critères de la première partie « Tetistre ». Dans cette partie, les insertions à gauches et à droites n'ont pas posé de problèmes particuliers, seulement un peu de complexité au niveau du chainage simple des pièces du plateau. Pour la fonction d'insertion d'une pièce à gauche sur le plateau, la complexité est : $\theta(1)$, tandis que celle d'insertion à droite est $\theta(n)$ ¹. La suppression de trois pièces consécutives, n'étant pas lié à première vue au chainage double des pièces (couleur et forme), n'a pas posé de grandes difficultés. La complexité de cette fonction de suppression est : $\Omega(1)$ dans le meilleur des cas et $O(n)$ ¹ dans le cas moyen.

Dans la seconde partie du projet « Tepatrice », il nous était demandé de nous intéresser aux décalages, via l'utilisation de listes de couleurs et de formes. Cette partie fut délicate, nous devions maintenant gérer le double chainage des pièces (forme et couleur). Après quelques essais infructueux, nous avons réussi à lier correctement le chainage simple et le double chainage de chaque pièce, selon notre configuration de départ. Ces fonctions de décalage à gauche ont une complexité $\theta(n)$. Avec le double chainage fonctionnel, nous avons pu reprendre notre fonction pour faire la suppression de trois pièces. Pour alléger le code, nous avons créé des sous-fonctions qui gèrent la suppression du double chainage.

Nous nous sommes ensuite attaqués à la gestion des fichiers de sauvegarde d'une partie en cours, ainsi que la mise en place d'un classement et le contrôle du score. Nous avons dû réfléchir à l'organisation de nos fichiers textes pour qu'il soit facile à manipuler par la suite. S'agissant de l'écriture dans le fichier

¹ n définit le nombre de pièces présentes sur le plateau.

du classement, nous avons souhaité faire au plus simple plutôt qu'au plus performant. En effet, nous utilisons un algorithme de tri (tri par insertion) qui n'est pas efficient pour des bases de données volumineuses. Cependant, le classement ne comporte qu'au maximum dix noms et dix scores, nous avons donc privilégié la compréhension à la performance. Cette fonction est donc $\theta(n^2)$.

Enfin, pour satisfaire la dernière partie du projet, nous avons affiché les cinq prochaines pièces à venir dans le jeu plutôt qu'une seule. Cette question nous a interrogé sur l'utilisation de l'allocation dynamique et les potentielles fuites de mémoire. Nous avons essayé de comprendre d'où pouvait venir les différentes fuites (fichier ouvert mais non fermé, pièce générée mais non supprimée à la fin du programme...). D'autre part, il nous était demandé de créer une interface graphique « simple ». Une section est dédiée à cette partie dans la suite de la documentation.

II. Quelques choix et améliorations proposés, d'autres envisagés

Nous allons présenter dans cette partie les améliorations et les choix que nous avons pris.

Tout d'abord, en raison d'une non-accessibilité aux différentes plateformes d'interface graphique, nous étions contraints de travailler en ASCII. Ainsi, nous avons développé une interface graphique afin que l'UI² du jeu soit accessible pour l'utilisateur. Nous avons utilisé l'affichage de codes Unicode pour remplacer les simples lettres. Nous avons recréé un logo (fait en ASCII) et implémenter quelques animations de chargement, avec des placements de texte notamment.

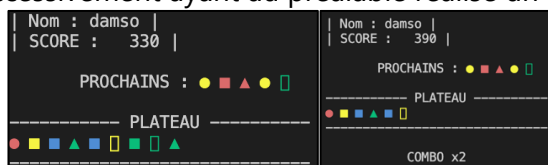
De plus, nous avons recréé un tableau des scores directement accessible dans le jeu avec un visuel attractif.

Au-delà des efforts réalisés sur l'interface graphique, nous avons aussi l'UX³ du jeu. Ainsi, nous avons implémenté des règles sur la saisie utilisateur (lorsqu'elle est demandée), mais aussi l'accessibilité aux différentes possibilités (démarrer une nouvelle partie, charger la précédente partie et voir le classement), le visuel du plateau du jeu...

Hormis ce que l'utilisateur peut voir, nous nous sommes efforcés, pour plus de lisibilité, d'organiser notre projet en plusieurs sous dossiers contenant un type particulier de fichier qui respectent tous une certaine norme. Les noms des fonctions sont tous en « *Pascal Case*⁴ » et ceux les macros en « *Screaming Snake Case*⁵ », les commentaires et le code sont écrits en français.

Concernant les améliorations pensées et mise en place pour certaines, nous avons instauré un système de combo pour les disparitions.

Voici les écrans s'affichant successivement ayant au préalable réalisé un DECALAGE LOSANGE.



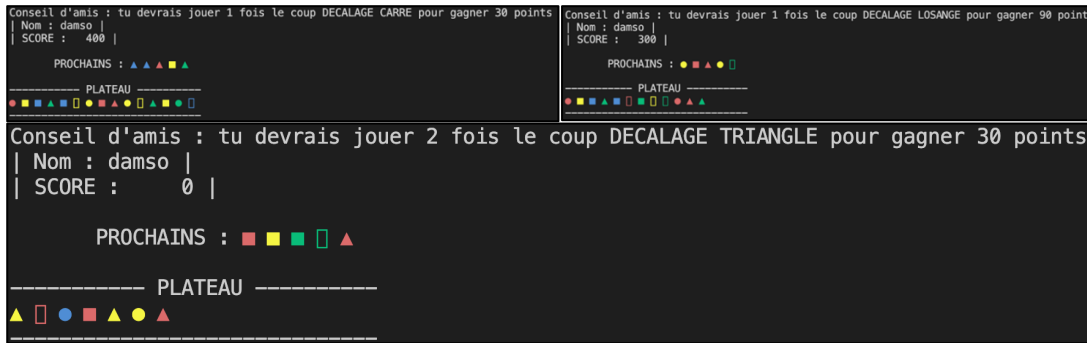
De plus, nous avons essayé d'implémenter un algorithme (ou IA) qui décrirait parmi une palette, une dizaine de coups « évidents », proposant ainsi à l'utilisateur le choix le plus pertinent. Vous trouverez ci-après des exemples de rendus :

² l'abréviation d'user interface ou interface utilisateur

³ l'abréviation d'user experience ou expérience utilisateur

⁴ une convention de nommage qui consiste à mettre tous les premières lettres de chaque mot en majuscule et de les coller

⁵ une convention de nommage qui consiste à relier les mots avec un underscore et à mettre toutes les lettres en majuscule



III. Interface graphique

Nous avons fait le choix d'utiliser la bibliothèque graphique MLV dont la documentation est disponible ici :

<https://www-igm.univ-mlv.fr/~boussica/mlv/api/French/html/index.html>

ou le guide d'installation est disponible ici :

<https://www-igm.univ-mlv.fr/~boussica/mlv/api/French/html/installation.html>

Ce choix a été motivé par plusieurs raisons :

- elle est plus simple d'utilisation que la bibliothèque SDL, qui était la bibliothèque vers laquelle nous sommes tournés en premier. Mais face à la difficulté d'apprentissage de son utilisation, nous avons cherché une alternative plus facile.

- Un grand nombre d'exemple d'utilisation des différentes fonctions de la bibliothèque sont mis à disposition sur le site

(<https://www-igm.univ-mlv.fr/~boussica/mlv/api/French/html/examples.html>), ce qui a facilité grandement l'apprentissage en autodidacte.

L'interface graphique a été faite en dernier. Le but de notre démarche a été de se débrouiller pour pouvoir l'intégrer au code déjà existant, en laissant intacte le plus possible la structure principale de notre code. Ceci s'est avéré être plus difficile que prévu. En effet, toute la structure et le schéma de pensée derrière notre code reposait sur un dialogue entre le joueur et le terminal. Toutes les informations du jeu étaient disponibles en temps réel dans le terminal, et le joueur devait rentrer une commande pour faire une action. Il a donc fallu en tout premier lieu, modifier la fonction `DeroulementJeu`, qui lançait l'action qui correspondait à la commande rentrée par l'utilisateur dans le terminal. Cette dernière était récupérée à l'aide d'un `scanf`, on mettait le résultat dans une variable et on testait sa valeur pour lancer l'action associée.

La première modification apportée a donc été de remplacer ce `scanf`. Pour cela, il y a la fonction [MLV_wait_keyboard](#) de MLV qui permet de récupérer la touche enfoncée par l'utilisateur dans la fenêtre qui a été créé par MLV. A la différence du `scanf`, l'utilisateur n'a pas besoin d'appuyer sur la touche Entrée de clavier.

Une fois qu'on a réussi à récupérer le souhait de l'utilisateur et à lancer l'action correspondante en conséquence, il a donc fallu s'attaquer au sujet principal de l'interface graphique, afficher le résultat des actions (c'est à dire l'état du plateau de jeu suite à une action) dans une fenêtre. Les pièces peuvent être dessinées grâce aux diverses fonctions comme [MLV_draw_circle](#), [MLV_draw_rectangle](#).

Le premier souci rencontré a été que le triangle et le losange n'étaient pas des figures pouvant directement être dessinées par une fonction associée. On peut les dessiner en utilisant la fonction [MLV_draw_filled_polygon](#), mais cela nécessite de donner en argument les coordonnées des points du polygone. Il a donc fallu calculer les différentes coordonnées de chaque point pour arriver à une figure avec une forme adéquate.

Une fois qu'on sait dessiner n'importe quelle pièce du plateau vient une autre difficulté, comment dessiner les pièces du plateau de manière à ce qu'elles forment une sorte de file sur l'écran ? En effet, dès qu'on dessine une pièce, il faut calculer la coordonnée x de la prochaine pièce et ainsi de suite, pour que toutes les pièces soient

plus ou moins régulièrement espacées les unes des autres. Cela n'a pas été facile pour toutes les pièces, au départ nous avions nos cercles qui empiétaient à moitié sur la pièce à leur gauche. Il a donc fallu définir plusieurs macros pour simplifier les calculs de positions des pièces : `TAILLE_PIECE`, `CENTRE_PIECE_SUR_PLATEAU` qui correspond à la ligne imaginaire qui doit "couper" chaque pièce sur le plateau en leur centre, ce qui facilite la représentation d'un plateau avec des pièces qui forment une sorte de file. Et enfin `ECART_CERCLE`, qui aide au calcul de comment doit être espacé un cercle par rapport à la pièce précédente. Tout cela nous a permis de régler le souci des cercles, et d'avoir une représentation d'un plateau de jeu dans la fenêtre.

En résumé, la principale difficulté rencontrée dans cette représentation du plateau a été le calcul des coordonnées des différents points.

Maintenant qu'on sait représenter un plateau de jeu, il reste encore à savoir comment représenter le plateau du jeu. L'astuce utilisée est très simple, elle a déjà été utilisée dans la fonction de sauvegarde dans un fichier de la partie, ainsi de dans nos fonctions de décalage. On parcourt le plateau, et on stock la forme et la couleur de chaque pièce (représentées par un caractère) dans un tableau de formes et un tableau de couleurs. A partir de là, on itère sur les deux tableaux et on utilise les fonctions de MLV pour dessiner les pièces correspondantes dans la fenêtre, tout en mettant à chaque fois à jour la position x de la prochaine pièce sur le plateau, de sorte à ce que les pièces successives forment une file indienne. Cette méthode nécessite de parcourir toute la liste chaînée, elle a donc un temps d'exécution linéaire. Tout ce processus correspond à la fonction `RecupPlateau` qui après avoir créé les tableaux de formes et couleurs, appellera `dessine_pieces_plateau` qui dessine les pièces dans la fenêtre correspondant à ces deux tableaux passés en argument.

Donc dans la fonction `DeroulementJeu`, pour chaque action qui modifiera le chainage des pièces, on appellera la fonction `RecupPlateau` qui réinitialisera la fenêtre et qui dessinera le nouveau plateau du jeu correspondant.

Pour dessiner les 5 pièces suivantes stockées dans le tableau `TabPieces`, on utilise le même procédé, sauf qu'on dessine cette deuxième file dans le coin droit de l'écran. Il faut juste changer l'axe y en conséquence.

Pour l'affichage du score, MLV permet la création de boîte de texte dans la fenêtre créée. A chaque action, nous appelons donc la fonction `AfficheScore` qui récupère la valeur numérique de la variable score, et qui affichera en haut à droite de la fenêtre "SCORE : valScore".

Nous avons rajouté des éléments esthétiques par la suite. Comme par exemple une musique (à l'aide de l'exemple :

https://www-igm.univ-mlv.fr/~boussica/mlv/api/French/html/advanced_203_music_8c-example.html, qui se lance au début du jeu et qui sera coupée une fois que la partie sera terminée (i.e si l'utilisateur a atteint les 15 pièces sur le plateau ou s'il fait le choix de sauvegarder).

Nous avons utilisé plusieurs images de fond. MLV permet de mettre une image en fond de la fenêtre (https://www-igm.univ-mlv.fr/~boussica/mlv/api/French/html/advanced_206_image_8c-example.html).

Pour rendre le jeu plus immersif, nous avons opté pour une image de borne d'arcade avec un écran noir sur lequel on dessinerait le plateau de jeu. A partir de cette image nous avons créé une image pour un écran au début du jeu qui simule une sorte de menu, un écran de sauvegarde et un écran de game over, qui lance une musique de game over et qui apparaît si l'utilisateur a posé 15 pièces sur le plateau. Toutes ces images ont été créées à partir du logiciel de retouche photo GIMP.

Même si on gagne en immersion, cela présente un défaut. A chaque modification du plateau suite à une action, aussi infime soit-elle, on doit remettre à zéro la fenêtre, redessiner l'image de fond, dessiner le nouveau plateau, dessiner les nouvelles 5 prochaines pièces, mettre à jour l'affichage du score. Ce sont des processus qui semblent assez lourd puisque nous avons quand même le temps de nous rendre compte d'un certain délai entre chaque action.

C'est sûrement pour cela que lorsqu'on lance ce programme avec `valgrind`, chaque action semble ralentir brièvement l'ordinateur (la musique se met à ralentir, l'écran se freeze légèrement...).

Nous pensons donc que c'est ici que se trouve le point d'amélioration principal de cette interface graphique, trouver un moyen de rendre l'actualisation des informations beaucoup moins lourde.

Pour finir, l'ajout de l'interface graphique semble avoir perturbé la gestion de la mémoire, indépendamment de notre volonté. Cela n'a pas l'air lié directement à notre code puisque la version sans interface à une gestion parfaite de la mémoire. Mais `valgrind` affiche de nombreuses erreurs qui seraient liées à des fonctions de SDL qu'utilise les fonctions de MLV. Nous ne savons concrètement pas si nous pouvons y faire quelque chose

L'ajout d'une interface graphique s'est avéré être une sacrée épreuve, avec du recul, nous aurions dû y penser

dès le départ, afin de structurer notre code en conséquence, plutôt que de la faire en dernier et d'essayer de la "greffer" à notre code déjà existant. Cependant, cela fut très enrichissant et notre jeu bénéficie grâce à elle d'une immersion non négligeable !

IV. Quelques mots pour conclure...

Nous espérons avoir pu détailler au mieux les contraintes posées par l'implémentation de certains passages, et les moyens mis en œuvre pour finalement y parvenir.

Nous avons trouvé ce projet très enrichissant. D'une part, parce qu'il reparaourt l'intégralité des points évoqués en C (et même plus loin !). D'autre part, les choix d'implémentation ont demandé de la recherche approfondie et de l'abnégation.

Merci pour votre lecture !

ROY Damien TD6

DESCHILDRE Matéo TD6